Q1. What is a Common Table Expression (CTE), and how does it improve SQL query readability?

- A Common Table Expression (CTE) in SQL is a temporary, named result set that exists only for the duration of a single SQL statement. It is defined using the `WITH` keyword and can be used within a `SELECT`, `INSERT`, `UPDATE`, or `DELETE` statement.

Example

Suppose we have an `Employees` table containing employee names, departments, and salaries. We want to find employees earning more than ₹50,000.

```
- WITH HighSalary AS (
-     SELECT First_name, Department, Salary
-     FROM Employees
-     WHERE Salary > 50000
- )
- SELECT *
FROM HighSalary;
```

Explanation:

1. `WITH HighSalary AS (...)` defines a CTE named `HighSalary`.
2. The CTE selects employees whose salary is greater than ₹50,000.
3. The main query retrieves the records from the CTE.

How does a CTE improve SQL query readability?

1. Simplifies complex queries: Breaks a large query into smaller, understandable parts.
2. Improves readability: Gives a meaningful name to a query result.
3. Reduces repetition: Allows the named result to be referenced multiple times within the same SQL statement.
4. Makes maintenance easier: Helps developers understand, modify, and debug queries.

Q2. Why are some views updatable while others are read-only? Explain with an example.

- A view in SQL is a virtual table created from the result of a `SELECT` query. Some views are updatable, meaning users can modify the underlying table through the view, while others are read-only, meaning data cannot be modified through them.
- **1. Updatable Views**
- An updatable view allows users to perform operations such as `INSERT`, `UPDATE`, and `DELETE` on the underlying table through the view.
- Example:

- CREATE VIEW HighSalaryEmployees AS
- SELECT Employee_id, First_name, Salary
- FROM Employees;
- To update an employee's salary through the view:
- UPDATE HighSalaryEmployees
- SET Salary = 60000
- WHERE Employee_id = 101;
- This updates the salary of employee 101 in the original `Employees` table.
- **2. Read-Only Views**
- A read-only view generally cannot be modified directly because its query structure makes it difficult for the database to determine which underlying rows should be changed.
- Example:
- CREATE VIEW DepartmentSalary AS
- SELECT Department, AVG(Salary) AS AverageSalary
- FROM Employees
- GROUP BY Department;
- This view calculates the average salary for each department. Updating `AverageSalary` through this view is generally not allowed because it is an aggregated value calculated from multiple employee records.
- **3. Difference Between Updatable and Read-Only Views**

Updatable View	Read-Only View
Allows data modification under supported conditions.	Does not allow direct data modification.
Usually based on a simple query involving one table.	Often involves aggregate functions, `GROUP BY`, or other complex operations.
Changes can affect the underlying table.	Primarily used to display or analyze data.

Q3. What advantages do stored procedures offer compared to writing raw SQL queries repeatedly?

- stored procedure in SQL is a set of SQL statements saved in the database that can be executed whenever needed. It helps avoid writing the same SQL queries repeatedly.

Advantages of Stored Procedures

1. Code Reusability: A stored procedure can be executed multiple times without rewriting the SQL code.
2. Improved Performance: Some database systems cache execution plans or otherwise optimize repeated procedure calls, which can reduce processing time.
3. Easy Maintenance: SQL logic is stored in one place, making it easier to update and manage.
4. Enhanced Security: Users can be granted permission to execute a procedure without giving them direct access to the underlying tables.
5. Reduced Network Traffic: Instead of sending multiple SQL statements separately, an application can call a single stored procedure.
6. Consistency: Using the same procedure ensures that the same business rules are applied whenever it runs.

Example

Suppose we frequently need to retrieve employees earning more than a specified salary.

```
- CREATE PROCEDURE GetHighSalaryEmployees
-     @MinSalary INT
- AS
- BEGIN
-     SELECT *
-     FROM Employees
-     WHERE Salary > @MinSalary;

END;
```

To execute the procedure:

```
EXEC GetHighSalaryEmployees @MinSalary = 50000;
```

The procedure retrieves employees earning more than ₹50,000 without requiring us to rewrite the query each time.

Note: This syntax is for SQL Server. Other database systems, such as MySQL, use slightly different syntax.

Q4. What is the purpose of triggers in a database? Mention one use case where a trigger is essential

- A trigger in a database is a special type of stored program that automatically executes when a specified event, such as `INSERT`, `UPDATE`, or `DELETE`, occurs on a table.

Purpose of Triggers

1. Automatic Execution: Performs actions automatically when specific database events occur.
2. Data Integrity: Helps maintain the accuracy and consistency of data.
3. Auditing: Records changes made to database records.
4. Business Rule Enforcement: Ensures that predefined rules are followed during data modifications.

Example: Employee Salary Audit

Suppose a company wants to maintain a record of every salary change made to its employees. A trigger can automatically store the old and new salaries in an audit table whenever an employee's salary is updated.

Example (SQL Server):

```
- CREATE TRIGGER SalaryAuditTrigger
- ON Employees
- AFTER UPDATE
- AS
- BEGIN
-     INSERT INTO SalaryAudit
-         (Employee_id, OldSalary, NewSalary)
-     SELECT
-         d.Employee_id,
-         d.Salary,
-         i.Salary
-     FROM deleted d
-     JOIN inserted i
-         ON d.Employee_id = i.Employee_id
-     WHERE d.Salary <> i.Salary;

END;
```

Explanation:

- `AFTER UPDATE` executes the trigger after an update operation.
- `deleted` contains the old employee records.
- `inserted` contains the updated employee records.
- The trigger records the old and new salaries when the salary changes.

One Essential Use Case

Auditing financial transactions: A trigger can automatically record changes to payment or salary records, helping organizations track modifications and investigate unauthorized changes.

Q5. Explain the need for data modelling and normalization when designing a database.

- Data modelling and normalization are important processes in database design. They help organize data efficiently, maintain accuracy, and reduce data duplication.

1. Data Modelling

Data modelling is the process of designing the structure of a database by identifying entities, attributes, relationships, and constraints.

Need for data modelling:

- Organizes data: Structures information into tables and columns.
- Defines relationships: Establishes relationships between tables using primary and foreign keys.
- Improves data integrity: Helps ensure that data is accurate and consistent.
- Simplifies maintenance: Makes databases easier to understand and manage.

Example: In a college database, we might create `Students`, `Courses`, and `Enrollments` tables and define relationships between them.

2. Normalization

Normalization is the process of organizing data into related tables to reduce unnecessary duplication and prevent data anomalies.

Need for normalization:

- Reduces redundancy: Avoids storing the same information repeatedly.
- Prevents anomalies: Reduces insertion, update, and deletion problems.
- Improves consistency: Ensures that changes to information are reflected appropriately.
- Simplifies data management: Makes records easier to maintain.

Example:

An unnormalized table might contain repeated student and course information:

Student_ID	Student_Name	Course
101	Rahul	SQL
101	Rahul	Python

102	Priya	SQL

After normalization, the data can be separated into `Students`, `Courses`, and `Enrollments` tables, reducing repeated information.

3. Difference Between Data Modelling and Normalization

Data Modelling	Normalization
Designs the overall database structure.	Organizes tables to reduce redundancy.
Identifies entities, attributes, and relationships.	Applies rules such as 1NF, 2NF, and 3NF.
Defines how data is connected.	Improves consistency and reduces anomalies.